NEXO / CHAT

UNIVERSIDAD DE CARTAGENA • UNIDAD 4

Comunicación en tiempo real

Proyecto en Node.js y Socket.IO

Descripción general

NEXO Chat es una aplicación web que permite intercambiar mensajes de manera instantánea entre varios participantes, sin recargar la página. Integra Node.js, el módulo HTTP nativo, Socket.IO, HTML, CSS y JavaScript. La interfaz permite ingresar con un apodo, visualizar participantes conectados, enviar mensajes e identificar quién está escribiendo.

ENTREGABLE	CONTENIDO
Código fuente	Servidor, cliente, estilos y archivos de configuración.
Documentación	README, desarrollo técnico y guion de presentación.
Pruebas	Casos automáticos y procedimiento de demostración entre dos pestañas.

Autor: _____ Fecha: _____

Entrega prevista: archivo .ZIP con un peso inferior a 10 MB. No incluye la carpeta node_modules; las dependencias se instalan mediante npm.

1. Objetivos y alcance

Objetivo general. Crear una aplicación web de chat en tiempo real, basada en Node.js y Socket.IO, que distribuya mensajes entre usuarios conectados de forma inmediata.

- Configurar un servidor HTTP en Node.js y enlazar Socket.IO.
- Transmitir eventos de envío, recepción, escritura y conexión.
- Presentar los mensajes y el estado de participantes en una interfaz adaptable.
- Aplicar validaciones, estructura de archivos clara y pruebas de integración.

2. Diseño de la solución

El navegador carga la interfaz por HTTP. Posteriormente, Socket.IO habilita la comunicación bidireccional. Cuando una persona escribe, el cliente emite un evento al servidor; el servidor valida y difunde el mensaje a todas las conexiones activas.

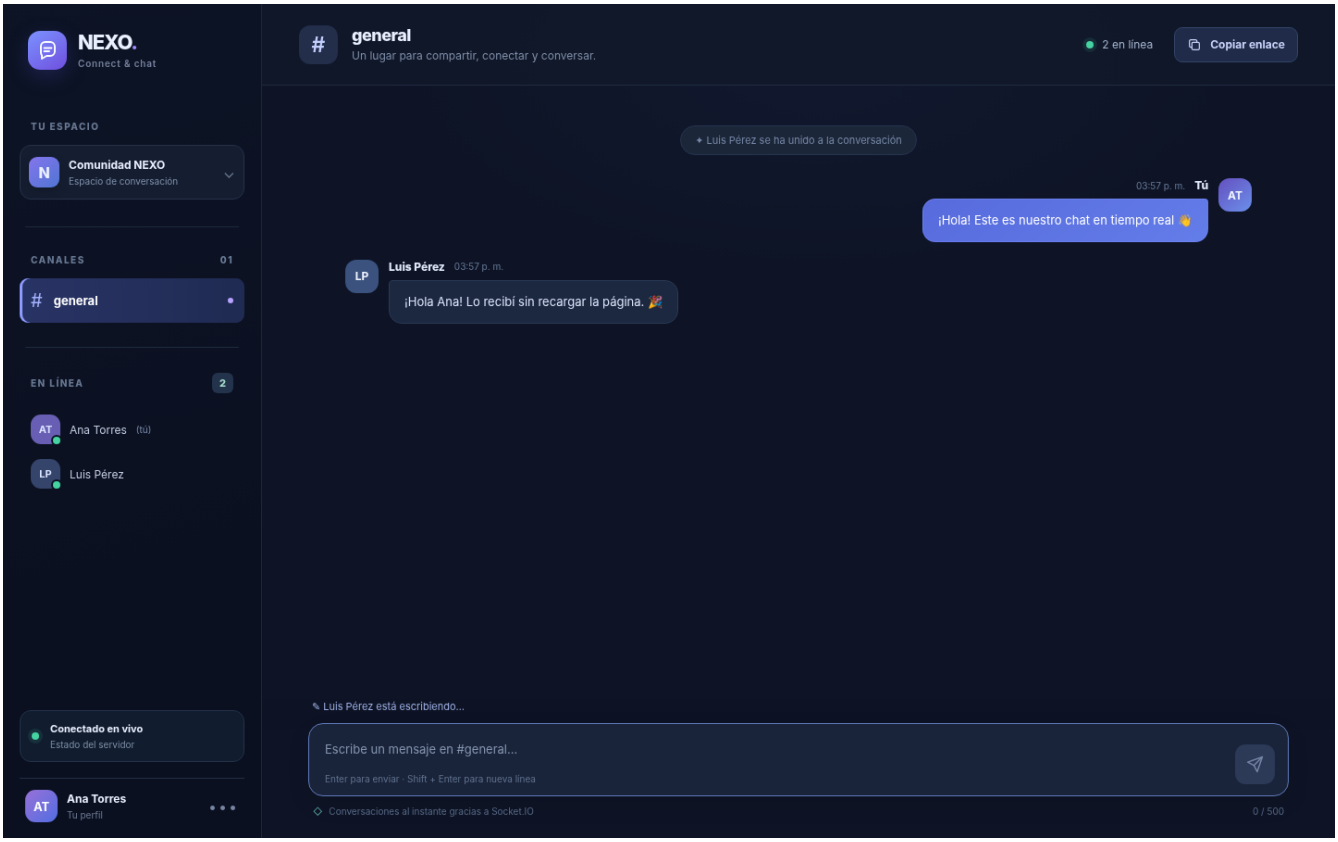
COMPONENTE	RESPONSABILIDAD
server.js	Servidor HTTP, validación de entradas, gestión de usuarios e historial temporal.
public/index.html	Estructura de la página, formularios y zonas para mostrar mensajes.
public/styles.css	Identidad visual, diseño responsive y accesibilidad.
public/app.js	Conexión al servidor, manejo de eventos y actualización del DOM.
tests/chat.test.js	Verificación programática de HTTP, mensajes, usuarios, historial y escritura.

3. Eventos importantes

EVENTO	ACCIÓN
user:join	El cliente envía su apodo y recibe confirmación de ingreso.
chat:send	El cliente transmite un mensaje para validación.
chat:message	El servidor difunde un mensaje a todos los clientes.
users:update	El servidor actualiza el listado de participantes.
chat:typing	Los participantes reciben el estado de escritura.
chat:history	El cliente recibe hasta 50 mensajes guardados temporalmente.

4. Interfaz de usuario

La experiencia visual incorpora un ingreso por apodo, un canal general, lista de personas conectadas, burbujas de mensajes, un indicador de escritura y una zona de composición. La interfaz se adapta a ventanas reducidas y dispositivos móviles.



Vista ilustrativa de la interfaz, generada con eventos simulados para revisar el diseño. No sustituye la evidencia de funcionamiento con Socket.IO real.

5. Validación y buenas prácticas

- Nombres de 2 a 24 caracteres y mensajes de 1 a 500 caracteres.
- Uso de textContent para presentar el contenido de usuarios como texto y no como HTML.
- Separación de archivos, funciones descriptivas y comentarios de apoyo.
- El historial es temporal y se borra al reiniciar el servidor.

6. Procedimiento de instalación

1. Descomprimir la carpeta NEXO-Chat del archivo ZIP.

2. Abrir la terminal dentro de la carpeta del proyecto.

3. Ejecutar npm install para descargar las dependencias de Socket.IO.

4. Ejecutar npm start para iniciar el servidor HTTP.

5. Abrir http://localhost:3000 en el navegador.

6. Abrir una segunda pestaña, ingresar con otro apodo y enviar mensajes entre ambas.

7. Ejecutar npm test para comprobar los casos de integración.

7. Plan de pruebas

CASO	RESULTADO ESPERADO
Dos usuarios	Ambos aparecen en la lista de participantes conectados.
Envío y recepción	El mismo mensaje aparece en ambas ventanas sin recarga.
Escritura	El segundo usuario observa el indicador correspondiente.
Nuevo participante	Recibe el historial reciente almacenado en memoria.
Mensaje vacío o largo	El servidor rechaza los mensajes no válidos.
Desconexión	Se reduce el número de participantes conectados.

Estado de verificación: se revisó la sintaxis, la representación de la interfaz en escritorio y móvil, y la lógica del servidor con eventos simulados. Las pruebas de integración que utilizan el paquete Socket.IO real deben ejecutarse en el equipo con las dependencias instaladas.

8. Conclusiones

El proyecto demuestra el patrón de comunicación basado en eventos y explica la función de Socket.IO como vínculo bidireccional entre servidor y clientes. El alcance se concentra en un canal general y un historial en memoria, suficientes para estudiar la interacción en tiempo real. Son posibles futuras ampliaciones con base de datos, autenticación y salas privadas.

Bibliografía

Socket.IO. Documentación v4: https://socket.io/docs/v4/

Socket.IO. Tutorial de chat: https://socket.io/get-started/chat/

Node.js. API HTTP: https://nodejs.org/docs/latest/api/http.html